



Banco de Dados

Prof. Elson de Abreu



Sistema gerenciador de banco de dados relacional

Microsoft SQL Server

Microsoft SQL Server

I. Bancos de Sistema:

- **master:** controla os bancos de usuários e a operação do SQL Server como um todo, mantendo controle de informações como contas de usuário, mensagens de erro do sistema etc.
- **model:** fornece um modelo para novos bancos de usuário.
- **tempdb:** fornece o espaço de armazenamento para tabelas temporárias e outras necessidades de armazenamento de trabalho temporário.
- **msdb:** fornece uma área de armazenamento para as informações de agendamento e o histórico de serviços. Utilizado para armazenamento de alertas e jobs (SQL Server Agent).

II. Bancos de Usuário

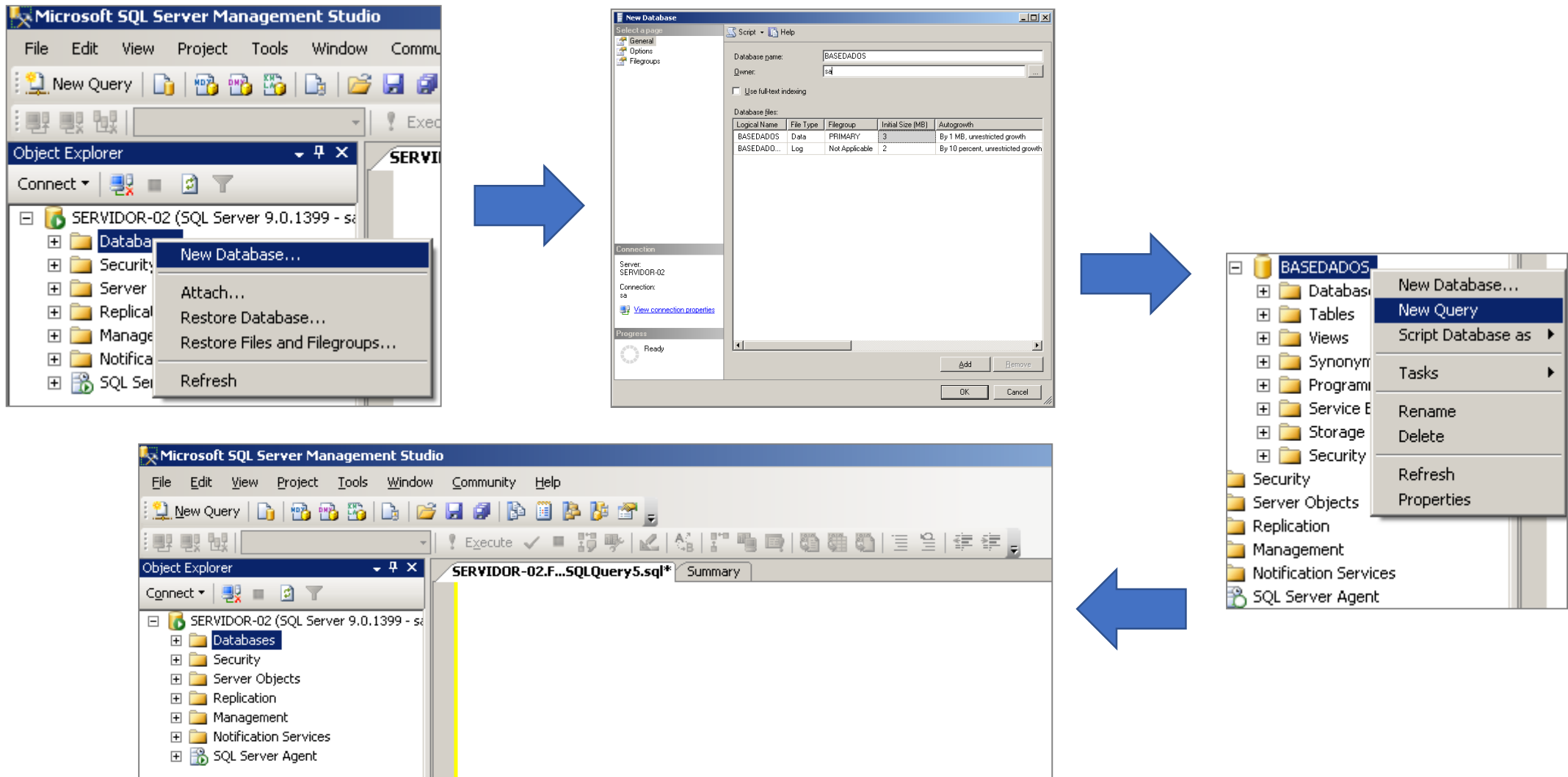
Objetos do banco de dados

- **Tabela:** coleção de registros que representam colunas associadas.
- **Tipos de Dados:** valores de dados permitidos para uma coluna ou variável; tipos de dados fornecidos pelo sistema; tipos de dados definidos pelo usuário.
- **Restrição (constraint):** define as regras referentes aos valores permitidos nas colunas; mecanismo padrão para forçar a integridade dos dados.
- **Default:** define um valor que será armazenado em uma coluna se nenhum outro valor for fornecido.
- **Regra (rule):** contém informações que definem valores válidos que são armazenados em uma coluna ou tipo de dados
- **Índice:** estrutura que fornece acesso rápido para a recuperação de dados; pode forçar a integridade dos dados

Objetos do banco de dados

- **Visão (view):** fornece uma maneira de examinar os dados de uma ou mais tabelas ou views de um banco.
- **Função definida pelo usuário (user-defined function):** utilizada para encapsular uma lógica executada frequentemente; retorna um valor.
- **Procedimento armazenado (stored procedure):** coleção nomeada de instruções Transact-SQL pré-compiladas, executadas em conjunto.
- **Gatilho (trigger):** forma especial de uma stored procedure que é executada automaticamente quando o usuário modifica os dados de uma tabela ou view.

Criando um novo banco de dados



Structured Query Language (SQL)

Daqui por diante , tentaremos utilizar a linguagem SQL implementada pela ferramenta MS SQL Server.

Linguagem de Consulta Estruturada

- Linguagem padrão para lidar com Bancos de Dados Relacionais.
- Originalmente chamada de SEQUEL, desenvolvida no Laboratório de Pesquisas da IBM na década de 70.
- Devemos lembrar que existe o SQL ANSI (linguagem padronizada) e as diversas implementações do SQL das inúmeras ferramentas disponíveis no mercado.
- **A depender da ferramenta (SGBDR), pode ser case sensitive!**

Divisões

- I. **DDL:** *Data Definition Language* engloba os comandos de definição do banco de dados, ou seja, interação com os objetos do banco. **(CREATE, ALTER, DROP)**
- II. **DML:** *Data Manipulation Language* corresponde aos comandos de manipulação dos dados. Composta de apenas três comandos, envolve interações de armazenamento, modificação, exclusão, inserção e atualização. **(INSERT, UPDATE, DELETE)**
- III. **DQL:** *Data Query Language* é o subconjunto responsável por comandos de consulta aos dados armazenados. Dentro dele, encontramos apenas o comando “**SELECT**”. É importante observar que em alguns materiais acadêmicos essa instrução aparece incorporada no conjunto DML.

Divisões

- IV. **DTL:** *Data Transaction Language* é um subconjunto do SQL para gerenciamento e controle de transação de dados. (**BEGIN/SET TRANSACTION, COMMIT, ROLLBACK, SAVEPOINT**)
- V. **DCL:** *Data Control Language* controla o acesso aos dados, tanto concedendo privilégio de acesso, quanto retirando a permissão do usuário ou usuária. Ou seja, esse subconjunto do SQL envolve comandos relacionados à segurança do banco de dados. (**GRANT, DENY, REVOKE**)

DDL: *create*

- O comando “CREATE” cria objetos, podendo ser usado para criar desde novos bancos de dados completamente zerados a tabelas específicas. Neste slide falaremos apenas da criação de tabelas, consulte as referências para os demais:

```
CREATE TABLE [database_name.[owner].|owner.]table_name
```

```
(column_name data_type [DEFAULT expressao] , ...)
```

- *Na cláusula CREATE TABLE, especifique o nome da tabela, em seguida especifique os nomes das colunas, seus tipos de dados e, caso exista, seu valor default .*
- *Nomes podem conter até 128 caracteres e devem começar com uma letra, _ ou #.*
- *Após o primeiro caractere, podem incluir letras, números ou símbolos _, @ ou #.*
- *Não deve duplicar o nome de outro objeto de propriedade do mesmo usuário e não deve ser uma palavra reservada do SQL Server.*

DDL: exemplo *create*

- Para confirmar a criação da tabela, execute o comando ***SP_HELP*** gerentes

```
CREATE TABLE gerentes (  
    codgerente INT IDENTITY(1,1),  
    nome          VARCHAR(20),  
    cargo         VARCHAR(20),  
    datacontratacao DATETIME DEFAULT GETDATE(),  
    salario        decimal(10,2)  
    CONSTRAINT pk_gerentes PRIMARY KEY (codgerente)  
)
```

Maiores dicas: <https://learn.microsoft.com/pt-br/sql/t-sql/statements/create-table-transact-sql?view=sql-server-ver16>

Alguns tipos de dados básicos

- Numéricos
 - Inteiros: int, smallint, tinyint, bigint, bit
 - Valores decimais precisos: decimal (p, [s]) ou dec (p, [s]), numeric(p, [s]), money, smallmoney
 - Valores de ponto flutuante: real, float[(p)]
- Datas
 - Datetime e Smalldatetime
- Caracteres
 - Tamanho fixo: char(n), nchar(n)
 - Tamanho variável: varchar(n), nvarchar(n)
- Binários
 - binary[(n)]
 - varbinary[(n)]
- Imagem e texto
 - image[(n)]
 - text[(n)]
 - ntext[(n)]
- Tipos de dados definidos pelo usuário
 - Baseados nos tipos de dados do sistema (domains)

DDL: *alter [table]*

- Utilize a instrução ALTER TABLE para:
 - Adicionar uma nova coluna.
 - Alterar o tipo de dados e precisão de uma coluna.
 - Remover colunas de uma tabela.
- **ALTER TABLE** table_name
ADD column_name data_type [DEFAULT expressao] , ...
- **ALTER TABLE** table_name
ALTER COLUMN column_name new_data_type
- **ALTER TABLE** table_name
DROP COLUMN column_name, ...

DDL: exemplo *alter [table]*

- Utilize a cláusula ADD para adicionar colunas a uma tabela

ALTER TABLE emp

ADD data_nascimento **DATETIME**, nome_pai **VARCHAR(50)**, nome_mae **VARCHAR(50)**

- As novas colunas serão as últimas da tabela!

DDL: *alter [table]*

- Utilize a cláusula ALTER COLUMN para alterar o tipo de dados e a precisão de uma coluna

ALTER TABLE *emp* ALTER COLUMN *nome_pai* VARCHAR(60)

- Por via de regra, somente uma coluna pode ser alterada por vez.
- A precisão de uma coluna só pode ser diminuída se a tabela estiver vazia ou se o tamanho de todos os dados da coluna forem menores ou iguais à nova precisão.
- O tipo de dados de uma coluna só poderá ser alterado se a tabela estiver vazia ou se todos os dados da coluna forem compatíveis com o novo tipo.

DDL: *alter [table]*

- Utilize a cláusula **DROP COLUMN** remover colunas de uma tabela:

ALTER TABLE *emp* **DROP COLUMN** *nome_pai, nome_mae*

- Utilize o comando **DROP CONSTRAINT**, como exemplo, para excluir uma constraint:

ALTER TABLE *dependentes* **DROP CONSTRAINT** *fk_dep_fun*

Onde fk_dep_fun é o nome da constraint e dependentes é a tabela a qual pertence a constraint.

- Mas para excluir uma tabela e todo seu conteúdo, a tabela não poderá estar sendo referenciada por outra tabela através da restrição de FOREIGN KEY, ou seja, a restrição deverá ser removida antes.

DROP TABLE *dependentes*

DML

- **INSERT:** incluir um novo registro em uma tabela.
- **UPDATE:** atualizar os dados de um ou mais registros em uma tabela.
- **DELETE:** deletar os dados de um ou mais registros de uma tabela.
- **SELECT INTO:** realiza uma consulta em uma tabela e cria uma nova com o resultado encontrado.

DML: insert

INSERT INTO <nome-tabela> [(<nome-coluna>, [<nome-coluna>])]

VALUES (<relação dos valores a serem incluídos>)

onde:

a) nome-tabela : representa o nome da tabela onde será incluída o registro.

b) nome-coluna : representa o nome da(s) coluna(s) terão conteúdo no momento da operação de inclusão.

Obs.: este comando pode ser executado de duas maneiras:

- 1) Quando todos os campos da tabela terão conteúdo - Neste caso não é necessário especificar as colunas, entretanto a relação dos valores a serem incluídos deverá obedecer a mesma sequência da definição da tabela.
- 2) Quando apenas parte dos campos da tabela terão conteúdo - Neste caso devem ser especificadas todas as colunas que terão conteúdo e os valores relacionados deverá obedecer esta sequência. Para os campos que não tem conteúdo especificado serão preenchidos com o valor NULL.

DML: insert exemplo

```
INSERT INTO funcionarios(codfunc, nome, cpf, salario) VALUES (1, 'Elson', '12345678900', 5000)
```

DML: update

UPDATE <nome-tabela>

SET <nome-coluna> = <novo conteúdo para o campo>

[<nome-coluna> = <novo conteúdo para o campo>]

WHERE <condição>

onde:

- a) nome-tabela : representa o nome da tabela cujo conteúdo será alterado.
- b) nome-coluna : representa o nome da(s) coluna(s) terão seus conteúdos alterados com o novo valor especificado.
- c) condição : representa a condição para a seleção dos registros que serão atualizados. Esta seleção poderá resultar em um ou vários registros. Neste caso a alteração irá ocorrer em todos os registros selecionados.

DML: update exemplo

- Todos os registros de funcionários cujo nome apareça “Silva” (seja no início, meio ou fim), seja trocado para “Silvio”

UPDATE funcionarios **SET** nome = 'Silvio' **WHERE** nome **LIKE** '%Silva%'

- Atualizando o campo “nome” e o “salario” do registro do funcionário cujo CPF seja “12345678900”:

UPDATE funcionarios **SET** nome = 'Elson de Abreu', salario = '54800' **WHERE** cpf **LIKE** '12345678900'

DML: delete

DELETE FROM <nome-tabela>

WHERE <condição>

onde:

- a) nome-tabela : representa o nome da tabela cujos registros serão deletados.
- b) condição : representa a condição para a exclusão dos registros. Este seleção poderá resultar em um ou vários registros.
Neste caso a operação irá ocorrer em todos os registros selecionados.

DML: delete exemplo

- Excluindo todos os registros da tabela de funcionários (sem nenhum critério):

DELETE FROM funcionarios

- Excluindo os registros de todos os funcionários cujo código (codfunc) seja maior ou igual a 10:

DELETE FROM funcionarios **WHERE** codfunc >= 10

DML: select into

SELECT <nome-coluna> **INTO** <nova-tabela>

FROM <tabela-atual>

WHERE <condição>

onde:

- a) nome-coluna : representa o nome da(s) coluna(s) selecionada(s).
- b) nova-tabela : representa o nome da nova tabela a ser criada com o resultado da consulta.
- c) tabela-atual : tabela onde residem os dados
- d) condição : representa a condição para a exclusão dos registros.

DML: select into exemplo

- Criar uma tabela contendo somente os campos Nome_Livro e IdAutor, correspondentes aos livros cujo Id é maior do que 105:

```
SELECT Nome_Livro, IdAutor  
INTO Livros_Autores  
FROM tbl_livros  
WHERE IdLivro > 105
```

Operadores lógicos

- = *igual a*
- > *maior que*
- >= *maior ou igual que*
- < *menor que*
- <= *menor ou igual que*
- <> *diferente*

- **Exemplos:**

- SELECT * FROM funcionarios WHERE codfunc > 5
- DELETE FROM funcionarios WHERE codfunc = 2

Outros operadores

- BETWEEN ... AND ... → *entre dois valores (inclusive)*
- IN (....) → *lista de valores*
- LIKE → *com um padrão de caracteres*
- IS NULL → *é um valor nulo*
- NOT → *negativo*

- **Exemplos:**

```
SELECT * FROM funcionarios  
WHERE codfunc BETWEEN 10 AND 20
```

```
SELECT * FROM funcionarios  
WHERE codfunc IN (5,7,10)
```

```
SELECT * FROM funcionarios  
WHERE nome LIKE 'e%'
```

```
SELECT * FROM funcionarios  
WHERE cpf IS NULL
```

```
SELECT * FROM funcionarios  
WHERE cpf IS NOT NULL
```

```
SELECT * FROM funcionarios  
WHERE codfunc NOT IN (5,7,10)
```

DQL

- O comando SELECT tem por objetivo selecionar um conjunto de registros em uma ou mais tabelas que atenda a uma determinada condição.

SELECT *ALL DISTINCT* <colunas> **FROM** <nome-tabela> [, <nome-tabela>]

WHERE <condição>

GROUP BY <nome-coluna>

HAVING <condição>

ORDER BY <nome-campo> **ASC DESC**

DQL: explicação das partes do comando

- **nome-tabela:** representa o nome da(s) tabela(s) que contem as colunas que serão selecionadas ou que serão utilizadas para a execução da consulta.
- **condição:** representa a condição para a seleção dos registros. Esta seleção poderá resultar em um ou vários registros.
- **nome-coluna:** representa a(s) coluna(s) cujos resultados são escolhidos para atender à consulta.
- **ALL:** opção default, mostra todos os valores obtidos na seleção.
- **DISTINCT:** opção que mostra os valores obtidos na seleção eliminando as duplicidades.
- **WHERE:** especifica o critério de seleção dos registros nas tabelas especificadas.
- **GROUP BY:** especifica os campos que serão agrupados para atender a consulta.
- **HAVING:** especifica uma condição para seleção de um grupo de dados. Esta opção só é utilizada combinada com a opção GROUP BY.
- **ORDER BY:** esta opção quando utilizada apresenta o resultado da consulta ordenado de forma crescente ou decrescente pelos campos definidos.

DQL: exemplo

- A partir da tabela de funcionários, selecionar codfunc, nome, cpf e salario, ou seja, todas as colunas independente do critério de pesquisa (selecionar todos os dados de todos os funcionários):

SELECT * FROM funcionarios

- Selecionando somente o cpf e nome de todos os funcionários:

SELECT cpf, nome **FROM** funcionarios

- Selecionando os nomes de funcionários ordenados de forma ascendente:

SELECT nome **FROM** funcionarios **ORDER BY** nome **ASC**

Utilize a seguinte estrutura para testar os comandos DQL de exemplo:

```
CREATE TABLE funcionarios (  
  codfunc INT NOT NULL,  
  nome VARCHAR(100) NOT NULL,  
  cpf CHAR(11) NULL,  
  salario DECIMAL(15,2) NOT NULL,  
  CONSTRAINT pkfuncionarios PRIMARY KEY (codfunc)  
)
```

DCL

- **GRANT:** concede aos usuários privilégios de acesso ao banco de dados.
- **DENY:** Nega uma permissão a uma entidade de segurança.
- **REVOKE:** retira os privilégios de acesso do usuário dados usando o comando GRANT.
- Como esta linguagem possui muitos detalhes, recomenda-se a leitura a partir do seguinte link:
<https://learn.microsoft.com/pt-br/sql/t-sql/statements/grant-transact-sql?view=sql-server-ver16>

DTL

- **COMMIT:** confirma uma transação.
- **ROLLBACK:** reverte uma transação no caso de ocorrer algum erro.
- **SAVEPOINT:** define um ponto de salvamento dentro de uma transação.
- **SET TRANSACTION:** especifica as características da transação.
- Estes comandos serão tratados com maior critério quando for abordado o assunto “transações”.